```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import MinMaxScaler,StandardScaler
from sklearn.decomposition import PCA
print("Libraries imported sucessfully:")
```

```
Libraries imported sucessfully:
```

```
data=pd.DataFrame({
    "study_hours":[2,4,5,7,8,10],
    "attendance_percent":[50,60,65,75,85,95],
    "assignment_score":[35,45,55,65,80,95],
    "final_marks":[38,48,56,68,82,94]
})
display(data)
```

```
features=data[["study_hours","attendance_percent","assignment_score"]]
print("Minimum Values:")
print(features.min())
print("Maximum Values:")
print(features.max())
```

```
Minimum Values:
study_hours            2
attendance_percent    50
assignment_score      35
dtype: int64
Maximum Values:
study_hours           10
attendance_percent    95
assignment_score      95
dtype: int64
```

```
features.plot(kind="box",figsize=(8,4))
plt.title("original features diffrent scales")
plt.ylabel("originla values")
plt.xticks(rotation=15)
plt.show
```

```
normalizer=MinMaxScaler()
normalized_array=normalizer.fit_transform(features)

normalized_df=pd.DataFrame(normalized_array,columns=features.columns,
                           index=features.index)
display(normalized_df.round(3))
```

```
print("Minimum Values after the normalization:")
print(normalized_df.min())
print("Maximum Values after the normalization:")
print(normalized_df.max())
```

```
Minimum Values after the normalization:
study_hours           0.0
attendance_percent    0.0
assignment_score      0.0
dtype: float64
Maximum Values after the normalization:
study_hours           1.0
attendance_percent    1.0
assignment_score      1.0
dtype: float64
```

```
normalized_df.plot(kind="box",figsize=(8,4))
plt.title("features after min-max normalization")
plt.ylabel("scaled values")
plt.xticks(rotation=15)
plt.show
```

```
standardizer=StandardScaler()
standardized_array=standardizer.fit_transform(features)

standardized_df=pd.DataFrame(standardized_array,columns=features.columns,
                             index=features.index)
display(standardized_df.round(3))
```

```
print("means after the standardization:")
print(standardized_df.mean().round(6))
print("Standard deviation after the standardization:")
print(standardized_df.std().round(6))
```

```
means after the standardization:
study_hours           0.0
attendance_percent   -0.0
assignment_score      0.0
dtype: float64
Standard deviation after the standardization:
study_hours           1.095445
attendance_percent    1.095445
assignment_score      1.095445
dtype: float64
```

```
standardized_df.plot(kind="box",figsize=(8,4))
plt.title("features after standardization")
plt.ylabel("standardized values")
plt.xticks(rotation=15)
plt.show
```

```
scaled_features=StandardScaler().fit_transform(features)
pca=PCA(n_components=2)
pca_array=pca.fit_transform(scaled_features)
pca_df=pd.DataFrame(pca_array,columns=["PC1","PC2"],index=features.index)
display(pca_df.round(3))
```

```
variance_ratios=pca.explained_variance_ratio_
for component,ratio in zip(["PC1","PC2"],variance_ratios):
  print(f"{component} explains {ratio:.2%} of the variance")
  total_retained=variance_ratio.sum()
  print(f"Total variance retained: {total_retained:.2%}")
```

```
PC1 explains 99.62% of the variance
Total variance retained: 99.96%
PC2 explains 0.34% of the variance
Total variance retained: 99.96%
```

```
component_name=["PC1","PC2"]
cumulative_variance=variance_ratios.cumsum()

plt.figure(figsize=(7,4))
plt.bar(component_names,variance_ratios)
plt.bar(component_names,cumulative_variance,marker="0")
plt.ylim(0,1.05)
plt.ylab("proportion of variance")
plt.xticks(rotation=15)
plt.show(
```